

LBG Project – Complete End-to-End Project Explanation

1. What exactly was the LBG project and what was its purpose?

LBG stands for **Lloyds Banking Group**, which is one of the major banking organizations in the UK.

The project was related to supporting and maintaining different digital banking applications used by customers for their day-to-day banking activities.

The main purpose of the project was to provide customers with secure, highly available, and reliable digital banking services.

2. What was the business use case of the project?

The business use case was similar to the normal services provided by a banking organization.

The platform supported different customer banking operations such as:

- Opening a new bank account
- Applying for personal or other types of loans
- Applying for a new credit card
- Managing existing accounts
- Viewing account balances
- Viewing transaction history
- Making payments and fund transfers
- Managing beneficiaries
- Receiving banking notifications
- Managing customer profile information

From the DevOps perspective, our responsibility was to ensure that these banking applications could be built, tested, deployed, monitored, and operated reliably.

3. What did the application/product do?

The application provided multiple digital banking services to customers.

For example, a customer could log in to the banking application and:

- Apply for a new loan
- Open a new bank account
- Apply for a credit card
- Check account balance
- View previous transactions
- Transfer money
- Add or manage beneficiaries
- Update personal information
- Access different banking services

The overall application was divided into multiple services or microservices, with each service responsible for a specific banking functionality.

4. Who were the end users?

The primary end users were **normal retail banking customers**, people like you and me who use banking services for everyday financial activities.

Depending on the module, there could also be internal bank employees, customer-support teams, operations teams, and other authorized business users.

Technical Project Explanation

5. What type of application were you handling?

We were mainly handling a **banking web application based on a microservices architecture**.

Different banking functionalities were divided into separate microservices.

For example:

- Customer Service
- Account Service
- Loan Service
- Credit Card Service
- Payment Service
- Transaction Service
- Notification Service
- Authentication Service

This helped teams develop, deploy, scale, and troubleshoot individual services independently.

6. What was the overall role of the DevOps team in the project?

The DevOps team was responsible for managing the complete application delivery lifecycle from developer code changes to production deployment.

Our major responsibilities included:

- CI/CD pipeline management
- Infrastructure provisioning
- Application deployments
- Docker image creation
- Kubernetes deployments
- Cloud infrastructure management
- Environment management
- Secrets management

- Monitoring and alerting
- Release management
- Production support
- Troubleshooting
- Automation
- Security and access management

Our objective was to make deployments automated, repeatable, secure, and reliable.

7. What was your specific role as a DevOps Engineer?

As a DevOps Engineer, my responsibilities included maintaining CI/CD pipelines, managing application deployments, working with Docker and Kubernetes, provisioning infrastructure through Terraform, managing cloud resources, monitoring applications, and supporting production issues.

I was also involved in:

- Creating and maintaining deployment pipelines
 - Managing Kubernetes workloads
 - Maintaining Helm charts
 - Managing environment-specific configurations
 - Troubleshooting failed deployments
 - Monitoring application and infrastructure health
 - Working with developers during release activities
 - Supporting production incidents
 - Automating repetitive operational tasks
 - Managing infrastructure using Terraform
-

8. Explain the complete end-to-end architecture from developer code push to production deployment.

A simplified flow was:

Developer

↓

Git Repository

↓

Feature Branch

↓

Pull Request

↓

Code Review

↓

CI Pipeline

↓

Build & Unit Testing

↓

Code Quality/Security Scan

↓

Docker Image Build

↓

Container Registry

↓

Deployment Pipeline

↓

Kubernetes Cluster

↓

Helm Deployment

↓

Load Balancer / Ingress

↓

Application Pods

↓

Database and Backend Services

The developer first worked on a feature branch and pushed the code to the Git repository.

After development was completed, the developer raised a Pull Request.

The code was reviewed by the development team and automated checks were executed.

Once the PR was approved and merged, the CI pipeline was triggered.

The pipeline performed activities such as:

Code Checkout → Build → Unit Testing → Code Quality Scan → Security Scan → Docker Image Build → Image Push

The Docker image was then pushed to the container registry.

After that, the deployment pipeline picked up the approved image version and deployed it to Kubernetes using Helm.

The same application version was gradually promoted through environments such as:

Dev → QA → UAT → Staging → Production

Production deployment normally required additional approvals and change-management checks.

9. Which environments were available?

We had multiple environments representing different stages of the software delivery lifecycle.

For example:

Dev → QA → UAT → Staging → Production

Dev was mainly used by developers for initial integration and validation.

QA was used by testing teams for functional and integration testing.

UAT was used for business-user validation.

Staging was designed to be very similar to production and was used for final validation.

Production was the live environment serving actual banking customers.

10. Were all environments in separate AWS/Azure accounts?

For better security and isolation, production was kept separate from non-production environments.

A typical structure could be:

Non-Production Account

- Dev
- QA

Pre-Production Account

- UAT
- Staging

Production Account

- Production

Production had stricter IAM permissions, networking controls, approval mechanisms, and security policies.

11. Where was the application hosted?

The application was hosted on the cloud platform.

A typical AWS-based architecture included services such as:

- Amazon EKS
- EC2
- ECR
- Application Load Balancer
- Route 53
- RDS
- S3
- CloudWatch
- IAM
- Secrets Manager
- KMS

- VPC

Most application workloads were containerized and deployed on Kubernetes.

12. Was Kubernetes used? What was the cluster architecture?

Yes, Kubernetes was used for running our containerized microservices.

For example, if Amazon EKS was being used:

Internet/User

↓

Route 53

↓

Application Load Balancer

↓

Ingress Controller

↓

Kubernetes Service

↓

Application Pods

The EKS control plane was managed by AWS.

Our application workloads were running on worker nodes.

We managed resources such as:

- Deployments
 - Services
 - ConfigMaps
 - Secrets
 - Ingress
 - HPA
 - Persistent volumes where required
-

13. Where did developers maintain their source code?

Developers maintained application source code in Git-based repositories.

For example, we can use **GitHub** as the project example.

Different microservices could have their own repositories, such as:

- customer-service
- account-service
- payment-service
- loan-service
- notification-service

Infrastructure code and deployment configurations could also be maintained separately.

For example:

- terraform-infrastructure
 - helm-charts
 - deployment-configurations
-

14. What Git branching strategy was followed?

A typical branching strategy was:

Feature Branch → Develop Branch → Release Branch → Main/Master Branch

For example:

A developer working on loan validation could create:

feature/loan-validation

The developer completed the changes, tested them locally, pushed the code, and raised a Pull Request against the develop branch.

15. What was the complete process when a developer created a new feature?

The process was generally:

Jira/User Story

→ **Feature Branch**

→ **Development**

→ **Unit Testing**

→ **Code Push**

→ **Pull Request**

→ **Code Review**

→ **Automated Validation**

→ **Merge**

→ **CI Pipeline**

→ **Dev Deployment**

Once testing was completed successfully, the same application version was promoted through higher environments.

16. How was code merged from the feature branch to develop/main?

Direct commits to important branches were restricted.

The developer first raised a Pull Request.

Before the code could be merged, different validations had to succeed, such as:

- Code review
- Unit tests
- Build validation
- Code quality checks
- Security checks

After receiving the required approvals, the PR was merged.

17. What input did the DevOps team receive after development was completed?

Depending on the deployment, the DevOps team could receive information such as:

- Repository name
- Branch or Git tag
- Application version
- Target environment
- Configuration changes
- Environment variables
- Secret requirements
- Database migration requirements
- Release ticket/change request
- Deployment instructions

However, much of the process was automated through CI/CD.

After the developer merged the code, the pipeline automatically handled most of the build and deployment activities.

18. What was the Pull Request and code-review process?

The developer raised a Pull Request after completing the feature.

The PR was reviewed by senior developers, technical leads, or designated reviewers.

Branch-protection policies prevented unauthorized direct changes.

Before merging, checks such as these had to pass:

PR Approval + Unit Tests + Build + Code Quality + Security Validation

Only after successful validation was the code merged.

19. How many control-plane and worker nodes were there?

If we were using Amazon EKS, the Kubernetes control plane was managed by AWS.

Therefore, we were not directly responsible for maintaining traditional Kubernetes master nodes.

For the worker layer, a production cluster could typically have around:

6–10 worker nodes, depending on workload and traffic.

Auto Scaling was enabled so that the number of nodes could increase or decrease based on demand.

20. What was the worker-node configuration?

An example worker-node configuration could be:

- Instance Type: `m5.xlarge`
- CPU: 4 vCPU
- Memory: 16 GB
- Operating System: Amazon Linux
- Minimum Nodes: 6
- Desired Nodes: 8
- Maximum Nodes: 15

The actual node type could vary between environments depending on application requirements.

21. How many Kubernetes clusters were there?

A realistic example could be:

Cluster 1: Dev + QA

Cluster 2: UAT + Staging

Cluster 3: Production

So approximately **three Kubernetes clusters** were managed.

Production had a dedicated cluster because it required stronger isolation, security, availability, and access controls.

22. How many microservices and pods were running?

Suppose the overall application had around **25–30 microservices**.

Each microservice normally had multiple pod replicas.

For example:

`payment-service` → 3 pods

`account-service` → 3 pods

customer-service → 2 pods

notification-service → 2 pods

In production, approximately **80–120 application pods** could be running depending on traffic.

Horizontal Pod Autoscaler could automatically increase or decrease replicas based on metrics such as CPU or memory utilization.

23. How were Load Balancer, Ingress, DNS, and the application connected?

The general request flow was:

Customer

↓

DNS / Route 53

↓

Application Load Balancer

↓

Ingress Controller

↓

Kubernetes Service

↓

Application Pods

Route 53 resolved the application domain.

The Application Load Balancer received customer requests.

Ingress routing rules identified which microservice should receive a particular request.

Kubernetes Services then forwarded requests to the appropriate application pods.

For APIs, the architecture could additionally include an API Gateway.

For example:

Customer/Mobile App

→ **API Gateway**

→ **Load Balancer**

→ **Ingress**

→ **Kubernetes Service**

→ **Pods**

24. Which database was used and how did the application connect to it?

A typical banking application could use a relational database such as **PostgreSQL or Oracle**, depending on the application.

If we use an AWS example, the architecture could be:

Application Pod

↓

Database Endpoint

↓

Amazon RDS PostgreSQL

Database credentials were never hardcoded inside the application source code.

Credentials and other sensitive information were stored securely using solutions such as **AWS Secrets Manager**.

The application retrieved the required credentials securely during runtime.

Production databases could also use:

- Multi-AZ deployment
 - Encryption at rest
 - Encryption in transit
 - Automated backups
 - Restricted security groups
 - Database monitoring
-

Interview-Ready 2-Minute Project Explanation

You can explain the project like this:

I worked on a Lloyds Banking Group project related to digital banking applications. The platform was used by retail banking customers for activities such as opening new accounts, applying for loans and credit cards, viewing transactions, making payments, and managing their banking services.

The application followed a microservices architecture and contained multiple services such as customer, account, payment, loan, transaction, and notification services.

Developers maintained their source code in Git repositories and worked using feature branches. Once development was completed, they raised Pull Requests. After code review and automated validation, the code was merged and the CI pipeline was triggered.

The pipeline performed code checkout, build, unit testing, code-quality and security checks, Docker image creation, and pushed the image to the container registry. The deployment pipeline then deployed the approved image to Kubernetes using Helm.

We had Dev, QA, UAT, Staging, and Production environments. Production was isolated from non-production with stricter security and access controls.

Traffic flowed through DNS, Load Balancer, Kubernetes Ingress, Services, and finally application pods. Backend services connected securely to databases, and credentials were managed through a secrets-management solution.

As a DevOps Engineer, I was mainly responsible for CI/CD pipelines, Kubernetes deployments, infrastructure automation using Terraform, cloud infrastructure, monitoring, release support, troubleshooting, and production deployments.